



Bosch eBike Systems

The smart system

Item: Live Data Interface



History of Changes

Version	Chapter	Change	Date	Department
1.0	All	Initial Release	01.05.2026	ECA2

Table of Content

1	Introduction.....	4
2	Technical Documentation.....	5
2.1	eBike Profile.....	5
2.1.1	Document Terminology	5
2.1.2	Introduction.....	6
2.1.3	Configuration	6
2.1.4	eBike Role Requirements.....	7
2.1.5	Accessory Role Requirements.....	7
2.1.6	Connection Establishment Procedures	11
2.1.7	Security Considerations	12
2.2	eBike Live Data Service	13
2.2.1	Introduction.....	13
2.2.2	Service Declaration	13
2.2.3	Service Characteristics.....	14
2.2.4	Forward Compatibility.....	15
2.3	References	17
2.4	ebike_live_data.proto	18
2.5	ebike_live_data.options	20
3	Terms of Use for the smart system's Live Data Interface	21
3.1	Preamble	21
3.2	Scope of Terms of Use.....	21
3.3	Use Contract.....	21
3.4	Rights of Use	21
3.5	Warranty and Liability	21
3.6	Service and Support	22
3.7	Data Protection	22
3.8	Applicable law and place of jurisdiction	22
3.9	Miscellaneous	22
4	Appendix	23

1 Introduction

Welcome to the documentation for the Bosch eBike Live Data Interface. We are pleased to offer a direct and open channel for your applications to connect with the smart system, enabling a new range of possibilities for riders and developers alike.

This interface provides third-party applications with read-only access to a comprehensive set of live data points via a standard Bluetooth Low Energy (BLE) connection. Available data includes:

- Speed
- Battery State of Charge (SOC)
- Rider Power
- Cadence
- Odometer
- Time
- Light Status
- Ambient Brightness
- Light Reserve
- System Lock
- Bike Not Driving
- Charger Connected
- Diagnosis Connected

Our goal is to foster a broad and creative ecosystem of compatible products. As such, access does not require a formal partnership or licensing agreement.

This document contains all the necessary technical specifications for implementation, along with the governing terms and conditions for the use of this interface. We look forward to seeing the innovative solutions you will create.

2 Technical Documentation

2.1 eBike Profile

Bluetooth® Profile Specification

2.1.1 Document Terminology

As specified by the Bluetooth SIG and the IEEE Standards Style Manual, the usage of the words "**shall**", "**must**", "**will**", "**should**", "**may**", and "**can**", is defined as follows:

- The word "**shall**" is used to indicate mandatory requirements strictly to be followed in order to conform to the standard and from which no deviation is permitted (shall equals is required to).
- The use of the word "**must**" is deprecated and shall not be used when stating mandatory requirements; must is used only to describe unavoidable situations.
- The use of the word "**will**" is deprecated and shall not be used when stating mandatory requirements; will is only used in statements of fact.
- The word "**should**" is used to indicate that among several possibilities one is recommended as particularly suitable, without mentioning or excluding others; or that a certain course of action is preferred but not necessarily required; or that (in the negative form) a certain course of action is deprecated but not prohibited (should equals is recommended that).
- The word "**may**" is used to indicate a course of action permissible within the limits of the standard (may equals is permitted).
- The word "**can**" is used for statements of possibility and capability, whether material, physical, or causal (can equals is able to).

2.1.2 Introduction

The eBike profile is used to enable an accessory device to obtain data from an eBike.

2.1.2.1 Profile Dependencies

This Profile requires the Generic Attribute Profile (GATT).

2.1.2.2 Bluetooth Specification Release Compatibility

This specification is compatible with any of the following:

- [Bluetooth Core Specification 4.2 or later \[1\]](#).

2.1.3 Configuration

2.1.3.1 Roles

The profile defines two roles:

- eBike
- Accessory

In this context, the eBike is a device that provides live data and states to an accessory.

2.1.3.1.1 GATT Roles - eBike

The eBike shall be a GATT server hosting the live data.

The eBike shall be a GATT client collecting device and/or battery information from the accessory, if supported by the accessory.

2.1.3.1.2 GATT Roles - Accessory

The accessory shall be a GATT client collecting the live data from the eBike.

The accessory may be a GATT server hosting device and/or battery information.

2.1.3.2 Profile - Service Relationships

The following table shows the relationships between services and the profile roles.

eBike GATT Server Services	Requirements	<->	Accessory GATT Server Services	Requirements
eBike Live Data Service	M: Mandatory		-	-
Device Information Service	O: Optional		Device Information Service	O: Optional
-	-		Battery Service	O: Optional

Table 2-1: Roles and Services

Unless otherwise noted a server may instantiate one and only one instance of the specified services.

2.1.3.3 Concurrency Limitations and Restrictions

This profile does not impose any concurrency limitations or restrictions for the accessory and the eBike.

In cases where there are multiple accessories or other Bluetooth components connected to the eBike, limitations or restrictions (e.g. connection, bandwidth, latency) might occur.

2.1.3.4 Topology Limitations and Restrictions

The accessory shall use the GAP peripheral role.
 The eBike shall use the GAP central role.

2.1.3.5 Transport Dependencies

This profile shall operate over an LE transport.

2.1.4 eBike Role Requirements

Note: Services and characteristics that are declared mandatory by the [Bluetooth Core Specification \[1\]](#), are not explicitly mentioned in this section.

2.1.4.1 eBike GATT Server Services

The eBike shall instantiate one and only one eBike Live Data Service.
 The eBike may instantiate one and only one Device Information Service.

GATT Server Services	eBike
eBike Live Data Service	M: Mandatory
Device Information Service	O: Optional

Table 2-2: eBike GATT Server Services

2.1.4.2 eBike GATT Client Services

The eBike may support the Device Information Service (as a client). The eBike may support the Battery Service (as a client).

GATT Client Services	eBike
Device Information Service	O: Optional
Battery Service	O: Optional

Table 2-3: eBike GATT Client Services

2.1.5 Accessory Role Requirements

Note: Services and characteristics that are declared mandatory by the [Bluetooth Core Specification \[1\]](#), are not explicitly mentioned in this section.

Note:

This profile is designed for dedicated accessory hardware. Smartphone operating systems may impose platform-specific restrictions on BLE Peripheral capabilities (e.g., limited control over advertising data content, or inability to perform GATT Client operations on connections initiated by a remote Central). These restrictions may prevent a smartphone application from fully conforming to the Accessory role requirements defined in this section. Smartphone support is not in scope of this specification.

2.1.5.1 Accessory GATT Server Services

The accessory may instantiate one and only one Device Information Service.
 The accessory may instantiate one and only one Battery Service.

GATT Server Services	Accessory
Device Information Service	O: Optional
Battery Service	O: Optional

Table 2-4: Accessory GATT Server Services

2.1.5.2 Accessory GATT Client Services

The accessory shall support the eBike Live Data Service (as a client).
 The accessory may support the Device Information Service (as a client).

GATT Client Services	Accessory
eBike Live Data Service	M: Mandatory
Device Information Service (DIS)	O: Optional

Table 2-5: Accessory GATT Client Services

2.1.5.3 Incremental eBike Live Data Service Requirements

This section describes additional eBike Live Data requirements beyond those defined in the eBike Live Data Service.

2.1.5.3.1 Service Solicitation UUIDs AD Type

While in a GAP Discoverable Mode for initial connection to an eBike, the accessory shall include the eBike Live Data Service UUID in the Service Solicitation UUIDs AD type field of the advertising data. For more information on Service Solicitation, see [\[2\], 1.10 Service Solicitation](#).

2.1.5.3.2 Local Name AD Type

For enhanced user experience, an accessory shall include the local name in its advertising data or scan response data.

2.1.5.3.3 Appearance AD Type

For an enhanced user experience, an accessory shall include the value of the appearance characteristic (*defined in [\[3\], 2.6 Appearance Values](#)*) in its advertising data or scan response data.

2.1.5.3.4 AD Flags

The accessory shall set the AD flags as follows:

- The discoverable mode flags set, depending on the selected discoverable mode
- BR/EDR Not Supported flag set to one
- Simultaneous LE and BR/EDR to Same Device Capable (Controller) flag set to zero

2.1.5.4 GATT Sub-Procedure Requirements

Requirements in this section represent a minimum set of requirements for an accessory. Other GATT sub-procedures may be used if supported by both client and server.

The table below summarizes additional GATT sub-procedure requirements beyond those required by all GATT Clients.

GATT Sub-Procedure	Accessory (Client) Requirements
Discover All Primary Services	C.1
Discover Primary Services by Service UUID	C.1
Discover All Characteristics of a Service	C.2
Discover Characteristics by UUID	C.2
Discover All Characteristic Descriptors	M
Read Characteristic Value	M
Notification	M
Read Characteristic Descriptors	M
Write Characteristic Descriptors	M
M: Mandatory C.1: Mandatory to support at least one of these sub-procedures. C.2: Mandatory to support at least one of these sub-procedures.	

Table 2-6: Accessory GATT Sub-Procedure Requirements

2.1.5.5 Service Discovery

The accessory shall perform a primary service discovery using either the GATT Discover All Primary Services sub-procedure or the GATT Discover Primary Services by Service UUID sub-procedure.

Important: The accessory shall not use persisted value handles or client characteristic configuration descriptors. A service and characteristic discovery shall be performed every time a connection has been established.

2.1.5.5.1 eBike Live Data Service Discovery

The accessory shall perform primary service discovery to discover the eBike Live Data Service.

2.1.5.5.2 Device Information Service Discovery

The accessory may perform primary service discovery to discover the Device Information Service.

2.1.5.6 Characteristic Discovery

As required by GATT, the accessory shall be tolerant of additional optional characteristics in the service records of services used with this profile.

2.1.5.6.1 eBike Live Data Service Characteristic Discovery

The accessory shall use either the GATT Discover All Characteristics of a Service sub-procedure or the GATT Discover Characteristics by UUID sub-procedure to discover the characteristics of the service.

The accessory shall use the GATT Discover All Characteristic Descriptors sub-procedure to discover the characteristic descriptors described in the following sections.

2.1.5.6.1.1 Live Data Characteristic

The accessory shall discover the Live Data characteristic.

The accessory shall discover the Client Characteristic Configuration descriptor of the Live Data characteristic.

2.1.5.6.2 Device Information Service Characteristic Discovery

The accessory may discover the characteristics of the Device Information Service.

In order for the accessory to discover the characteristics of the Device Information Service, it shall use either the GATT Discover All Characteristics of a Service sub-procedure or the GATT Discover Characteristics by UUID sub-procedure.

2.1.5.7 Live Data

The accessory shall control the configuration of notifications (i.e., via the Client Characteristic Configuration descriptor) of the Live Data characteristic.

The accessory shall be able to receive and process multiple notifications of the Live Data characteristic from the eBike, including multiple notifications within a single connection event.

2.1.5.7.1 ATT_MTU Requirements

Both the eBike and the accessory shall support an ATT_MTU of at least 247 bytes.

The accessory (as GATT Client of the eBike Live Data Service) shall initiate an ATT_MTU exchange (ATT_EXCHANGE_MTU_REQ) after connection establishment, requesting an ATT_MTU of at least 247 bytes. The eBike (as GATT Server of the eBike Live Data Service) shall respond to the ATT_EXCHANGE_MTU_REQ with a Server RX MTU of at least 247 bytes.

The negotiated ATT_MTU shall be large enough to encode the values of all Live Data fields into a single ATT notification (i.e., a single ATT_HANDLE_VALUE_NTF PDU without fragmentation at the ATT layer). This requirement also ensures headroom for future extensions of the Live Data characteristic while maintaining backwards compatibility with existing accessory implementations.

If the negotiated ATT_MTU is less than 247 bytes, the eBike shall disconnect the link using HCI disconnect reason 0x13 (Remote User Terminated Connection).

2.1.5.7.2 Data Length Extension Requirements

Both the eBike and the accessory shall support the LE Data Length Extension (DLE) feature.

Both the eBike and the accessory shall support a Link Layer Data PDU payload size of at least 251 bytes (MaxTxOctets and MaxRxOctets).

The eBike shall initiate a Data Length Update procedure (LL_LENGTH_REQ) after connection establishment, requesting a PDU payload size consistent with the required ATT_MTU (e.g., 251 bytes for an ATT_MTU of 247 bytes, accounting for the 4-byte L2CAP header).

If the effective Link Layer Data PDU payload size remains below 251 bytes after the Data Length Update procedure, the eBike shall disconnect the link using HCI disconnect reason 0x13 (Remote User Terminated Connection).

2.1.5.7.3 Device Information Service Characteristics

The accessory may read the value of Device Information Service characteristics.

2.1.6 Connection Establishment Procedures

This section describes the procedures for establishing and terminating connections that are used by an eBike and its accessories in typical scenarios.

2.1.6.1 Accessory Connection Establishment Procedures

This section describes connection procedures that an accessory should follow to initiate a connection with an eBike.

- Section 2.1.6.1.1 describes the connection procedure that is used when the accessory is not bonded with the eBike.
- Section 2.1.6.1.2 describes the connection procedure that is used when the accessory is bonded with the eBike.
- Section 2.1.6.1.3 describes the connection procedure that is used when the established connection is broken after a link loss and a reconnection is required.

2.1.6.1.1 Connection Procedure for Unbonded Devices

This procedure is used for connection establishment when the accessory is not bonded with an eBike and ready for connection (for example, when the accessory is initially powered on or a dedicated mode for establishing a connection with a (new) eBike is triggered).

If a connection is not established within a certain period of time, which is defined by the manufacturer, the accessory either may continue sending background advertising, to reduce power consumption, for as long as it chooses, or it may stop advertising. The advertising interval and the time to perform advertising are implementation specific and should be configured with consideration for user expectations of connection establishment time using the GAP timers defined in [\[1\] Volume 3, Part C, Section 9.3.11](#).

If a connection is not established within a time limit defined by the accessory, the accessory may exit the GAP Connectable Mode.

Table 2-7 summarizes the recommended connection procedure if the accessory is not bonded to an eBike.

Recommended GAP Modes	Recommended Filter Policy
Limited Discoverable Mode	Attempt to connect to any eBike
Undirected Connectable Mode	
Bondable Mode	

Table 2-7: Recommended Connection Procedure for Unbonded Devices

The accessory shall support the bonding mode and procedure.

2.1.6.1.2 Connection Procedure for Bonded Devices

Recommended GAP Modes	Recommended Filter Policy
Non-Discoverable Mode	Attempt to connect only bonded eBikes in the Filter Accept List.
Undirected Connectable Mode	

Table 2-8: Recommended Connection Procedure for Bonded Devices

If a connection is not established within $T_{GAP}(adv_fast_period)$, the accessory may either continue sending background advertising to reduce power consumption for as long as it chooses, or stop advertising. The advertising interval and the time to perform advertising are implementation specific and should be configured with consideration for user expectations of connection establishment time using the GAP timers defined in [\[1\] Volume 3, Part C, Section 9.3.11](#).

If a connection is not established within a time limit defined by the accessory, the accessory may exit the GAP Connectable Mode.

When the accessory is disconnected and the accessory is ready for reconnection (e.g., when commanded by the user), the accessory should reinitiate the connection procedure (e.g., start advertising).

2.1.6.1.3 Link Loss Reconnection Procedure

When a connection is terminated due to link loss, the accessory should attempt to reconnect to the eBike by entering the GAP Connectable Mode.

2.1.7 Security Considerations

All eBike Live Data Service characteristics require encryption with security mode 1 and level 2, specifically Just Works with LE Secure Connections.

The accessory shall accept the LE Security Mode and Security Level combination requested by the eBike.

2.2 eBike Live Data Service

Bluetooth® Service Specification

2.2.1 Introduction

This service allows a client to collect live data and states of an eBike (server).

2.2.1.1 Service Dependency

This service does not depend upon any other services.

2.2.1.2 Bluetooth Specification Release Compatibility

This specification is compatible with any of the following:

- [Bluetooth Core Specification 4.2 or later \[1\]](#).

2.2.1.3 GATT Sub-procedure Requirements

Requirements in this section represent a minimum set of requirements for a server. Other procedures may be used if they are supported by both the client and the server.

Table 2-9 summarizes additional GATT sub-procedures required beyond those required by all GATT servers.

GATT Sub-Procedure	Requirements
Notifications	M: mandatory
Read Characteristic Descriptors	M: mandatory
Write Characteristic Descriptors	M: mandatory

Table 2-9: Additional GATT sub-procedure requirements

2.2.1.4 Transport Dependencies

This service shall operate over an LE transport.

2.2.1.5 Application Error Codes

This service does not define any attribute protocol application error codes.

2.2.1.6 Byte Transmission Order

All characteristics in this service are encoded using protobuf [\[4\]](#) (see Section 2.2.3.1), which defines its own byte transmission order as part of its wire format. The standard Bluetooth little-endian byte order therefore does not apply to the characteristic values in this service.

2.2.2 Service Declaration

This service shall be instantiated as a primary service.

This service and all its characteristics share the same base UUID 0000xxxx-aaa2-11e9-81b4-2a2ae2dbcce4, abbreviated as ebike_uuid(xxxx) throughout this document.

This service shall be identified by ebike_uuid(eb20).

2.2.3 Service Characteristics

The following characteristics shall be exposed. Unless otherwise specified, only one instance of each characteristic is permitted within this service. Where a characteristic can be notified or indicated, a client characteristic configuration descriptor shall be included in that characteristic as required by the [Core Specification \[1\]](#).

Characteristic Name	Requirement	Properties (mandatory)	Properties (optional)	UUID	Security Permissions
Live Data	O: Optional	Read, Notify	-	ebike_uuid(eb21)	Encryption

Table 2-10: Service Characteristics

Note: Individual characteristics are marked as optional to allow future extensions of this service with additional characteristics. A client is not required to support all characteristics and may subscribe only to those it is interested in.

2.2.3.1 Service Characteristics Content Format

The service characteristics shall be encoded using Protocol Buffers (protobuf) [\[4\]](#) as defined in the following files:

- [ebike_live_data.proto](#) — message definitions
- [ebike_live_data.options](#) — nanopb [\[5\]](#) generator options (optional, for nanopb-based implementations)

The protobuf schema shall use proto3 syntax with a minimum language revision of 3.15 to support the optional keyword for explicit field presence tracking.

Implementations may use any protobuf library that is compatible with the proto3 wire format (e.g., nanopb, protobuf-c, Google protobuf). The choice of protobuf library is implementation specific and shall not affect interoperability, as all compliant libraries produce and consume the same wire format.

2.2.3.2 Live Data

The Live Data characteristic value shall be encoded as a `com.bosch.ebike.LiveData` protobuf message (referred to as `LiveData` throughout this document) as defined in [ebike_live_data.proto](#).

When a client reads the Live Data characteristic, the server shall return the latest values of all available `LiveData` fields.

When notifications are enabled, the server shall send a notification whenever one or more `LiveData` fields change. The notification shall contain at least the fields that have changed since the last notification. A notification may also include additional unchanged fields ([see Section 2.2.4.3](#)).

Note: The server should send no more than one notification per connection event ([see Section 2.2.4.5](#)). Delivery of a notification may be delayed by one or more connection events due to RF interference, link layer scheduling, or concurrent connections on the eBike. The client shall not assume that notifications are delivered within a specific time frame.

Note: The service does not provide notifications on the availability of all individual data sources. For example, when a battery is disconnected from the system, the server does not notify the client that the `battery_soc` field is no longer available. The last transmitted value remains unchanged with no indication that it may be stale. The client shall not assume that a previously received field value is still valid as the underlying data source might have become unavailable. A future version of this service may introduce dedicated fields to indicate data source availability (e.g., similar to `charger_connected`). Note that the existing proto3 field presence accessors (e.g., `has_*` in C implementations) cannot be used for this purpose, as field absence in a notification already indicates that the field value has not changed ([see Section 2.2.4.3](#)).

2.2.4 Forward Compatibility

This section defines requirements to ensure that clients remain compatible with future versions of this service. The `LiveData` protobuf message may be extended with additional fields in future service versions. Clients shall be designed to gracefully handle such extensions without loss of functionality.

2.2.4.1 Protobuf Forward Compatibility

Proto3 provides built-in forward compatibility mechanisms that this service relies on:

- **Unknown fields:** When a protobuf decoder encounters a field tag it does not recognize (i.e., a field added in a newer version of the `.proto` file), it shall skip the field without error. Proto3 decoders preserve this behavior by default. The client shall not treat unknown fields as a decoding error.
- **Optional field presence:** This service requires proto3 release 3.15 or above (see Section 3.1) to support the `optional` keyword, which provides explicit field presence tracking via field presence accessors (e.g., `has_*` in C implementations). The client shall use field presence information to distinguish between a field that is absent (not included in the notification) and a field that is present with its default value.
- **Field ordering:** The client shall not assume a specific encoding order of fields within a protobuf message, as the protobuf wire format does not guarantee field ordering.

2.2.4.1.1 Implementation-Specific Considerations

The protobuf library used may impose constraints on forward compatibility. Implementors shall verify that their chosen library meets the requirements of this section.

Nanopb [5]: Nanopb may use statically allocated structures with compile-time size limits. The `.options` file defines maximum sizes for variable-length fields (e.g., `max_size`, `max_count`) and integers (e.g. `uint8_t`). By default, nanopb discards unknown fields during decoding. This is acceptable for this service, as the client is only required to process fields it recognizes. Implementors shall ensure that the nanopb `.options` file is kept in sync with the `.proto` file for each supported service version.

Other libraries: Libraries that use dynamic memory allocation (e.g., Google protobuf, protobuf-c with allocators) typically handle unknown fields and message extensions without additional configuration. Implementors shall ensure that their library is configured to skip — rather than reject — unknown fields during decoding.

2.2.4.2 Enum Handling

Future service versions may extend existing enums with new values. The client shall handle unrecognized enum values gracefully:

- The client shall not treat an unrecognized enum value as a decoding error.
- The client should treat an unrecognized enum value as an unsupported or unknown state and fall back to a safe default behavior.
- In proto3, unrecognized enum values are preserved as their integer representation during decoding. Most protobuf libraries (including nanopb) store them as-is in the corresponding integer field.

Important: Enum fields shall not be removed or renumbered in future versions of the `.proto` file. New values shall only be appended.

2.2.4.3 Unknown and Unchanged Fields

The client shall ignore any `LiveData` fields that it does not recognize. This allows the server to introduce new fields without breaking existing client implementations.

The client shall not rely on the presence of unchanged fields in a notification. A field is considered unchanged if its value has not been modified since the last notification. The server may include such fields in a notification for convenience, but their presence is not guaranteed across notifications or across service versions.

2.2.4.4 Buffer Sizing

The client and server shall size their protobuf encode and decode buffers based on the maximum ATT notification payload, not on the theoretical maximum encoded size of the `LiveData` message as defined in the current service version. The theoretical maximum encoded size may be smaller than the available ATT payload in the current version, but future versions may add fields that utilize the full payload capacity.

The maximum ATT notification payload is `ATT_MTU - 3` bytes (accounting for the 1-byte ATT opcode and 2-byte attribute handle in the `ATT_HANDLE_VALUE_NTF` PDU). With the required `ATT_MTU` of 247 bytes, this results in a maximum payload of **244 bytes**.

Encode and decode buffers shall be allocated for at least 244 bytes. This ensures that the implementation can handle future extensions of `LiveData` that use the full available payload without requiring buffer reallocation or recompilation.

2.2.4.5 Multiple Notifications per Connection Event

Future versions of this service may require the server to send multiple notifications per connection event (e.g., if the Live Data characteristic is extended with additional fields that exceed the `ATT_MTU`, or if partial updates are sent at a higher frequency).

To ensure forward compatibility, the client shall be able to receive and process multiple notifications of the Live Data characteristic within a single connection event.



2.3 References

- [1]: Bluetooth Core Specification v4.2 or later
- [2]: Bluetooth Core Specification Supplement
- [3]: Bluetooth Assigned Numbers
- [4]: Google Protocol Buffers (<https://developers.google.com/protocol-buffers>)
- [5]: Nanopb — Protocol Buffers with small code size (<https://github.com/nanopb/nanopb>)

2.4 ebike_live_data.proto

```
// SPDX-Copyright-Text: 2026 Robert Bosch GmbH
// SPDX-License-Identifier: Apache-2.0
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//      http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

syntax = "proto3";

package com.bosch.ebike;

enum LightState {
    LIGHT_STATE_INVALID = 0;
    LIGHT_STATE_OFF = 1;
    LIGHT_STATE_ON = 2;
}

message LiveData {
    /*
     * The bike speed in km/h.
     * Unit: 1/100 km/h
     * Range: 0 to 655.35 km/h
     */
    optional uint32 speed = 1;
    /*
     * The cadence of the rider.
     * Unit: 1 rpm
     * Range: -32768 to 32767 rpm
     */
    optional int32 cadence = 2;
    /*
     * The present power of the rider.
     * Unit: 1 W
     * Range: 0 to 65535 W
     */
    optional uint32 rider_power = 5;
    /*
     * The ambient brightness in lux.
     * Unit: 1/1000 lux
     * Range: 0 to 4294967.295 lux
     */
    optional uint32 ambient_brightness = 9;
    /*
     * The battery state of charge.
```



```
* Unit: 1 %
* Range: 0 to 100 %
*/
optional uint32 battery_soc = 10;
/*
* The current time (UTC).
* Unit: 1 second (since 1.1.1970)
* Range: -9223372036854775808 to 9223372036854775807 seconds
*/
optional int64 time = 11;
/*
* The total distance in meters.
* Unit: 1 meter
* Range: 0 to 4294967295 meters
*/
optional uint32 odometer = 12;

// The bike light state.
optional LightState bike_light = 17;
// True, if the system is in a locked state.
optional bool system_locked = 21;
// True, if the charger is currently connected. This doesn't necessarily mean that the bike is currently
charging (e.g. when battery is already full).
optional bool charger_connected = 22;
// True, if the light reserve state is active.
optional bool light_reserve_state = 23;
// True, if the Diagnosis Tool is currently connected to the bike. This is usually the case at the bike
dealer/repair center.
optional bool diagnosis_program_active = 24;
// True, if bike is standing still. The bike might still be slightly moving, depending on the sensor
accuracy.
optional bool bike_not_driving = 25;
}
```



2.5 ebike_live_data.options

```
com.bosch.ebike.LiveData.speed int_size:IS_16  
com.bosch.ebike.LiveData.cadence int_size:IS_16  
com.bosch.ebike.LiveData.rider_power int_size:IS_16  
com.bosch.ebike.LiveData.ambient_brightness int_size:IS_32  
com.bosch.ebike.LiveData.battery_soc int_size:IS_8  
com.bosch.ebike.LiveData.odometer int_size:IS_32
```

3 Terms of Use for the smart system's Live Data Interface

3.1 Preamble

The terms of use for the smart system's live data interface (hereinafter referred to as the "Terms of Use") govern the use of the the smart system's live data interface including any technical documentation thereto (hereinafter referred to as the "Interface") provided by Robert Bosch GmbH (hereinafter referred to as "Bosch").

3.2 Scope of Terms of Use

Bosch provides the Interface on a fully public and open basis with equal access for all users with a business interest. The access and use of the Interface are free of charge. The specification is available for anyone to download and use without requiring registration, login, or the provision of any personal data (e.g., email address, company name). There are also no specific keys or other technical barriers required to access the Interface at a software level. Bosch grants access to the Interface and allows the use of it exclusively based on these Terms of Use.

3.3 Use Contract

3.3.1 By using the Interface a use contract based on these Terms of Use between Bosch and the user will be agreed.

3.3.2 Bosch will endeavor to provide an uninterruptible availability of the Interface in the scope of its technical and operational possibilities, but there will be in any case no right of the user to have such an uninterruptible availability. Bosch does not and cannot warrant that the access to the Interface will not be impaired by downtime, maintenance activities, further developments, updates and upgrades or malfunctions. Technical malfunctions (e.g. interruption in the electricity supply, hardware and software errors, and technical problems in the data lines) also can, however, give rise to temporary restrictions or interruptions.

3.3.3 Bosch reserves the right to change, to amend or to close the Interface at any time.

3.4 Rights of Use

Bosch grants the user a non-exclusive, non-transferable, non-sublicensable royalty-free right to use the Interface to develop, test, and offer products and services that are compatible with the smart system. The user is not authorized to modify the Interface without the prior written consent of Bosch. Bosch retains all other rights to the Interface, in particular trade name, proprietary knowledge and other intellectual property rights pertaining to the Interface.

3.5 Warranty and Liability

3.5.1 Subject to change, Bosch confirms that the Interface is compatible with all Bosch eBike systems equipped with a Bosch eBike remote control (BRC) of the smart system.

3.5.2 As use of the Interface is free of charge, it is provided as is. Bosch makes no representations or warranties of any kind, whether express, implied, statutory, or otherwise, regarding the Interface. This includes, but is not limited to, any warranties of merchantability or fitness for a particular purpose.

3.5.3 Bosch is liable in accordance with the statutory provisions

- a) In the event of intent or gross negligence;
- b) In accordance with any applicable mandatory law;
- c) Within the scope of a guarantee given by Bosch; and
- d) For loss of life or bodily injury/harm.

Furthermore, Bosch's liability is excluded.

3.6 Service and Support

Bosch does not provide any assistance or service with regard to implementation or in case of any technical problems by using the Interface.

3.7 Data Protection

The user is solely responsible for any processing of personal data that may occur in connection with the use of the Interface. Should any processing of personal data occur, the user shall comply with any applicable data protection regulations.

In this context, the user and Bosch act as separate controllers. Bosch acts neither as a processor nor as a joint controller.

3.8 Applicable law and place of jurisdiction

3.8.1 The use contract shall be exclusively governed by the laws of the Federal Republic of Germany without giving effect to its conflict of law provisions. The application of the UN Convention on Contracts for the International Sale of Goods (CISG) is excluded.

3.8.2 The exclusive place of jurisdiction for any legal disputes arising from or in connection with the use contract in accordance with these Terms of Use shall be Stuttgart, Germany.

3.9 Miscellaneous

3.9.1 Bosch reserves the right to change these Terms of Use at any time.

3.9.2 Any translation of these Terms of Use into other languages are for the user's information only. In case of conflicts between different language versions of these Terms of Use, the English language version will prevail.

3.9.3 Should any provision of these Terms of Use is or becomes void, the remaining provisions remain unaffected.

4 Appendix

Release Notes - smart system control unit v19 - Live Data Interface

Release notes for the smart system control unit software with respect to the Live Data Interface

Copyright © 2026 Robert Bosch GmbH. All rights reserved.

- **smart system control unit release:** v19
- **applicable live data interface release:** v1.0.0

1 Summary

eBike system release v19 is the first release to provide **experimental** support for the Live Data Interface (v1.0.0), introducing Bluetooth LE connectivity between eBikes and accessory devices. The eBike acts as a GAP Central and GATT Server, providing live data to connected accessories via the eBike Live Data Service.

Experimental: This is the first eBike software release implementing the Live Data Interface specifications. The implementation may be subject to changes, bug fixes, or behavioral adjustments in future software releases. See Section 6 for known issues and limitations.

Key capabilities:

- Real-time live data streaming from eBike to accessory via GATT notifications
- Protocol Buffers (proto3) encoding for compact, forward-compatible data serialization
- ATT_MTU and LE Data Length Extension support (see Section 6 for current limitations)
- LE Secure Connections (Security Mode 1, Level 2) with mandatory bonding
- Forward compatibility for future data field extensions without breaking existing accessories

2 Supported Accessory Appearance Categories

2.1 Appearance Value Structure

The Bluetooth Appearance is a 16-bit unsigned integer transmitted in little-endian byte order (as per the standard Bluetooth byte ordering defined in the Bluetooth Core Specification [\[1\]](#), Vol. 3, Part C, Section 11). Its bit-field structure is defined in the Bluetooth Core Specification Supplement [\[2\]](#), Part A, Section 1.12:

Bits	Field	Width	Description
15–6	Category	10 bit	Device category (e.g., 3 = Watch)
5–0	Subcategory	6 bit	Subcategory within category (0 = Generic)

The category and subcategory can be encoded and decoded as follows:

- $\text{enc_appearance}(\text{category}, \text{subcategory}) = (\text{category} \ll 6) \mid \text{subcategory}$
- $\text{dec_category}(\text{appearance}) = \text{appearance} \gg 6$
- $\text{dec_subcategory}(\text{appearance}) = \text{appearance} \& 0x3F$

Example — Sports Watch (category 3, subcategory 1):

- Encode: $\text{enc_appearance}(3, 1) = (3 \ll 6) | 1 = 0x00C0 | 0x01 = 0x00C1$
- Decode: $\text{dec_category}(0x00C1) = 0x00C1 \gg 6 = 3$ (Watch), $\text{dec_subcategory}(0x00C1) = 0x00C1 \& 0x3F = 1$ (Sports Watch)

2.2 Supported Categories

The eBike UI matches on the **category** only (i.e., $\text{dec_category}(\text{appearance})$). All subcategories within a supported category are represented with the same icon.

Appearance Category	Category	Matched Range	Notes
Watch	3	0x00C0 – 0x00FF	e.g., Sports Watch (0x00C1)
Display	5	0x0140 – 0x017F	
Cycling	18	0x0480 – 0x04BF	

Accessories with an appearance not listed above will be shown with a generic icon in the eBike UI.

3. New Features

This being the initial release, all items below are new.

- **eBike Live Data Service:** Single-characteristic GATT service (UUID 0000eb20-eaa2-11e9-81b4-2a2ae2dbcce4) providing live data via notifications and read
- **Protobuf Encoding:** Live Data characteristic encoded using Protocol Buffers proto3 (revision 3.15+), with optional field presence semantics
- **ATT_MTU Negotiation:** The eBike supports an ATT_MTU of 247 bytes and responds accordingly to ATT_MTU exchange requests from accessories. See Section 6 (LDI-003) for current limitations regarding disconnect behavior.
- **Data Length Extension:** See Section 6 (LDI-001, LDI-003) for current DLE limitations.
- **Bonding & Reconnection:** Mandatory bonding with reconnection procedures for bonded devices and link loss recovery
- **Service Solicitation:** Accessories advertise the eBike Live Data Service UUID for directed discovery
- **Optional Accessory Services:** Device Information Service and Battery Service optionally hosted by the accessory
- **Forward Compatibility:** Protobuf-based extensibility — unknown fields and enum values are gracefully handled

4. Changes

Not applicable — initial release.

5. Bug Fixes

Not applicable — initial release.

6. Known Issues and Limitations

ID	Issue
LDI-001	DLE not requested by eBike
LDI-002	Unchanged fields in Live Data notifications
LDI-003	No disconnect on insufficient MTU or DLE

- **LDI-001 — DLE not requested by eBike**

The eBike does not initiate a Data Length Update procedure (LL_LENGTH_REQ) after connection establishment, despite the requirement in eBike Profile [Section 2.1.5.7.2](#)

Workaround: The accessory (client) can initiate the Data Length Update procedure instead.

- **LDI-002 — Unchanged fields in Live Data notifications**

In this release, the server includes various unchanged fields alongside changed fields in each Live Data notification. The set of unchanged fields included is not guaranteed and may change in future releases.

Workaround: Clients shall ignore unchanged fields and shall not rely on their presence in notifications. See eBike Live Data Service [Section 2.2.4.3](#)

- **LDI-003 — No disconnect on insufficient MTU or DLE**

The eBike does not validate the negotiated ATT_MTU (≥ 247) or the effective LE Data Length (≥ 251) after connection establishment. No automatic disconnection occurs when these values are below the required thresholds.

The eBike supports an ATT_MTU of 247 bytes. However, if the accessory does not initiate an ATT_MTU exchange or requests a smaller value, the negotiated MTU remains at the default (23 bytes). In that case, the BLE stack silently truncates notification payloads to 20 bytes, resulting in a corrupted protobuf byte stream on the accessory.

Similarly, if DLE remains at the default (27-byte LL payloads), larger ATT PDUs are fragmented across multiple Link Layer packets, increasing latency and RF overhead.

In both cases the connection remains active and the eBike continues sending (potentially corrupted) notifications. See: eBike Profile [Sections 2.1.5.7.1](#) and [2.1.5.7.2](#)

Workaround: Future releases will enforce the specified ATT_MTU and DLE thresholds and disconnect the link (HCI reason 0x13) if the requirements are not met. Accessory implementors shall ensure their devices support $\text{ATT_MTU} \geq 247$ and $\text{DLE} \geq 251$ and shall test the disconnection behavior.

7 Compatibility

7.1 Bluetooth Core Specification Compatibility

Bluetooth Core Specification 4.2 or later [\[1\]](#)

7.2 Backwards Compatibility

Future releases are intended to maintain backwards compatibility. The use of Protocol Buffers (unknown field skipping, optional field presence, append-only enum values) is designed to support this goal, though breaking changes may occur in exceptional cases and will be communicated in the corresponding release notes.

8. Upgrade Instructions

Not applicable — initial release